In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data
from skimage.transform import resize

# Load cameraman image (already grayscale)
image = data.camera()
image = resize(image, (256, 256))  # Resize for faster FFT (optional)

# Get image size
M, N = image.shape

# Generate frequency grid (centered)
u = np.fft.fftfreq(M).reshape(-1, 1)
v = np.fft.fftfreq(N).reshape(1, -1)

# Compute Laplacian filter in frequency domain
laplacian_filter = -4 * (np.pi ** 2) * (u**2 + v**2)

# Apply 2D FFT
F = np.fft.fft2(image)

# Apply Laplacian filter in frequency domain
F_lap = F * laplacian_filter

# Inverse FFT to get spatial result
laplacian_image = np.fft.ifft2(F_lap).real

# Plot results
plt.figure(figsize=(12, 5))

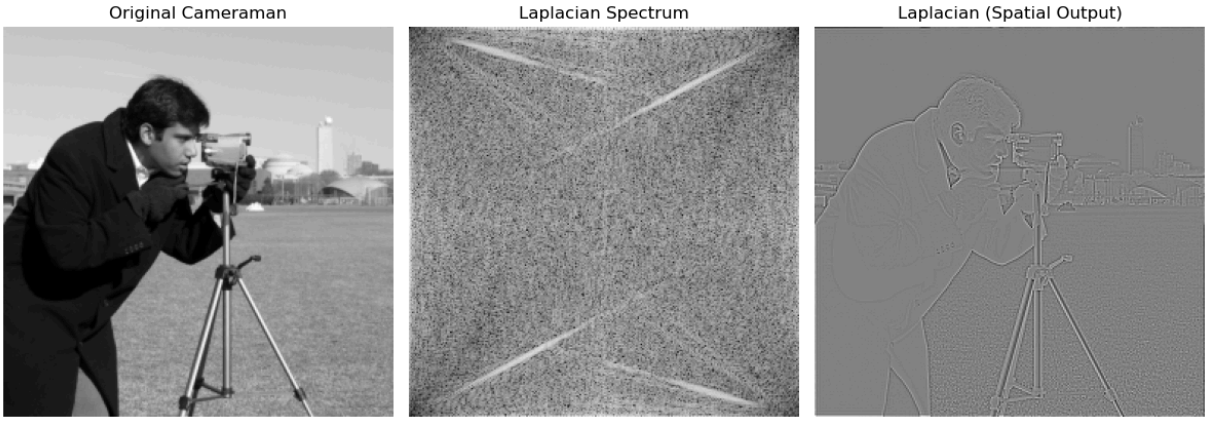
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Cameraman')
plt.axis('off')

plt.subplot(1, 3, 2)
plt.imshow(np.log(1 + np.abs(F_lap)), cmap='gray')
plt.title('Laplacian Spectrum')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.imshow(laplacian_image, cmap='gray')
plt.title('Laplacian (Spatial Output)')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Loading [MathJax]/extensions/Safe.js



In [2]:
```python
from numpy.fft import fft2, ifft2, ifftshift

# Load and resize image
image = data.camera()
image = resize(image, (256, 256))  # Resize for faster FFT (optional)

# Sobel filters
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]], dtype=float)

sobel_y = np.array([[-1, -2, -1],
                    [ 0,  0,  0],
                    [ 1,  2,  1]], dtype=float)

# Proper padding and centering of kernel in spatial domain
def center_embed_kernel(kernel, shape):
    padded = np.zeros(shape)
    kh, kw = kernel.shape
    ph, pw = shape
    # Find center coordinates
    cy, cx = ph // 2, pw // 2
    # Insert kernel centered at the image center
    padded[cy - kh//2:cy - kh//2 + kh, cx - kw//2:cx - kw//2 + kw] = kernel
    return fft2(ifftshift(padded))  # Shift before FFT to place (0,0) at top-left

# Compute FFT of image
F = fft2(image)

# FFT of centered Sobel filters (use the provided function)
H_x = center_embed_kernel(sobel_x, image.shape)
H_y = center_embed_kernel(sobel_y, image.shape)

# Multiply in the frequency domain
G_x = F * H_x
G_y = F * H_y

# Inverse FFT to get spatial results
sobel_x_image = ifft2(G_x).real
sobel_y_image = ifft2(G_y).real

# Compute combined gradient magnitude
```

Loading [MathJax]/extensions/Safe.js

```
sobel_magnitude = np.sqrt(sobel_x_image**2 + sobel_y_image**2)

# Plot results
fig, axes = plt.subplots(2, 3, figsize=(14, 8))

# Row 1: Original Image and Frequency Domain Spectrums
axes[0, 0].imshow(image, cmap='gray')
axes[0, 0].set_title('Original Cameraman', fontsize=12)
axes[0, 0].axis('off')

axes[0, 1].imshow(np.log(1 + np.abs(G_x)), cmap='gray')
axes[0, 1].set_title('Sobel X Spectrum: log(1 + |G_x|)', fontsize=12)
axes[0, 1].axis('off')

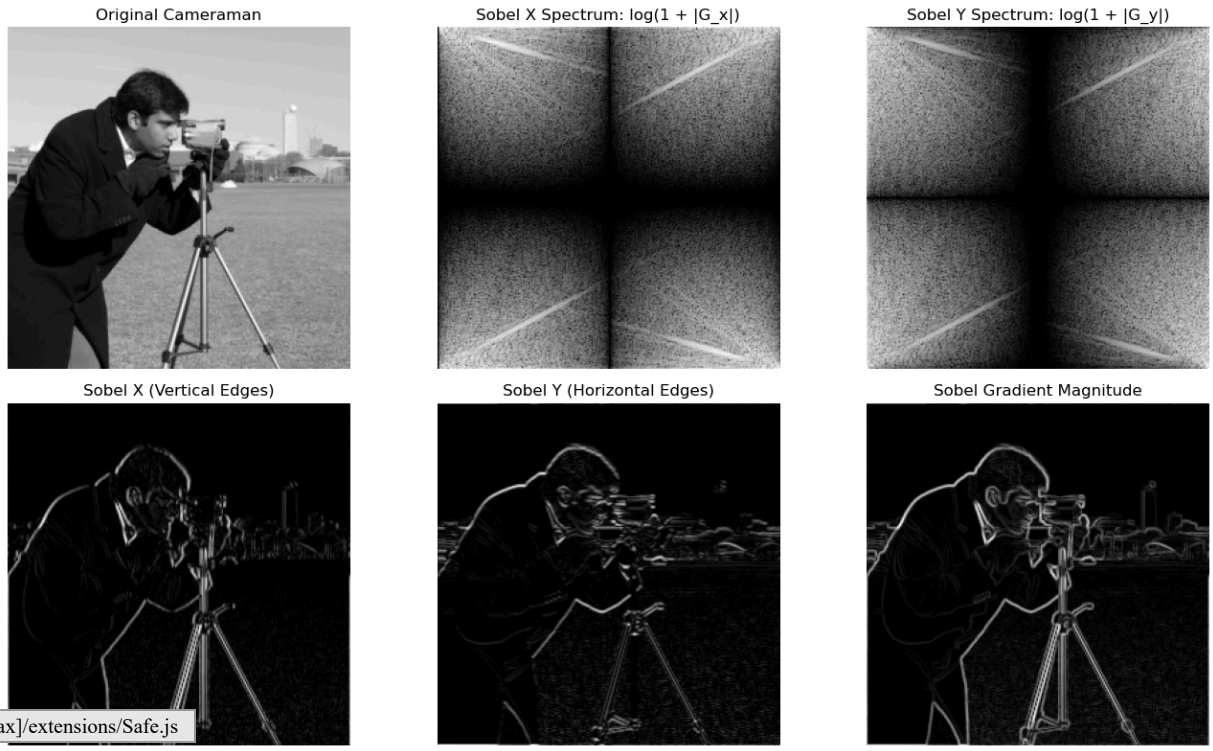
axes[0, 2].imshow(np.log(1 + np.abs(G_y)), cmap='gray')
axes[0, 2].set_title('Sobel Y Spectrum: log(1 + |G_y|)', fontsize=12)
axes[0, 2].axis('off')

# Row 2: Spatial Domain Filter Outputs
axes[1, 0].imshow(np.abs(sobel_x_image), cmap='gray')
axes[1, 0].set_title('Sobel X (Vertical Edges)', fontsize=12)
axes[1, 0].axis('off')

axes[1, 1].imshow(np.abs(sobel_y_image), cmap='gray')
axes[1, 1].set_title('Sobel Y (Horizontal Edges)', fontsize=12)
axes[1, 1].axis('off')

axes[1, 2].imshow(sobel_magnitude, cmap='gray')
axes[1, 2].set_title('Sobel Gradient Magnitude', fontsize=12)
axes[1, 2].axis('off')

plt.tight_layout()
plt.show()
```



Loading [MathJax]/extensions/Safe.js

Loading [MathJax]/extensions/Safe.js